nara-kosensai

システム構成・仕様書(印刷版)

対象リポジトリ: `nara-kosensai/manifest`(旧 `config`)、`fumikun/2026kosensai`
最終更新: 2026-08-29

目次

第 1 章 概要 2
第 2 章 アーキテクチャ 3
2.1 リクエスト経路 3
2.2 namespace 構成 6
2.3 Ingress ルーティング(旧 nginx からの移植) 6
第 3 章 ネットワーク・セキュリティ 7
3.1 公開ポート 7
3.2 TLS / mTLS 7
3.3 RBAC 7
第 4 章 シークレット管理 9
第 5 章 CI/CD パイプライン 10
第 6 章 監視 11
第 7 章 障害と設計判断 12
7.1 nftables ルールのインターフェーススコープ(2026-08-29) 12
7.2 VPS 再起動によるホスト名変化と k3s ノード名の固定(2026-08-28) 12
第 8 章 リファレンス 13

第 1 章　概要

nara-kosensai は、単一 VPS 上のシングルノード k3s クラスタで運用されている。旧構成(VPS ネイティブの nginx と複数の docker-compose)から全面移行したもので、外部通信は原則 Cloudflare Tunnel 経由のみとし、クラスタはノードの公開インターフェースに極力ポートをバインドしない。唯一の例外が本番アプリ (`nara-kosensai.com` / `www.nara-kosensai.com`)で、Cloudflare の Origin Authentication(mTLS)を使い、Tunnel を経由せず直接オリジンに到達させる。

このクラスタの設定は 4 つのリポジトリに分散して管理する。特定サービスに紐づかない共有プラットフォーム設定・クラスタブートストラップは本リポジトリ(`nara-kosensai/manifest`)に、アプリ/コンテンツ固有の設定はそれぞれのサービスのリポジトリに置く方針である(表 1.1)。

リポジトリ	内容
`nara-kosensai/manifest` (本リポジトリ)	クラスタブートストラップ(`k3s/`)、特定サービスに紐づかない共有プラットフォーム設定 (`k8s/` :cloudflared, Grafana/Loki/Prometheus, 過去年度アーカイブ, storybook, kosensai-form-preview のデプロイ定義)、この仕様書
`fumikun/2026kosensai`	当年アプリ本体(backend/frontend/db/pgadmin/website)の Dockerfile・k8s マニフェスト (`infra/k8s/base/`, `overlays/{prod,test}`)・CI/CD
`nara-kosensai/www`	過去年度(2017–2025)アーカイブサイトの中身とそのイメージビルド CI。デプロイ先マニフェストは本リポジトリ側が保持
`fumikun/kosensai-form-preview`	高専祭掲載情報プレビューツール本体・CI。デプロイ先マニフェストは本リポジトリ側

表 1.1：設定リポジトリの役割分担

リポジトリ名について：本リポジトリは GitHub 上で `nara-kosensai/config` から `nara-kosensai/manifest` へリネーム済み(2026 年)。旧 URL(`.../config.git`)は自動転送されるため既存クローンは動作し続けるが、新規クローン・CI は新 URL を使うのが望ましい。

第 2 章　アーキテクチャ

2.1　リクエスト経路

外部からのリクエストは、宛先ホストによって 2 つの異なる経路のいずれかを通る。両方とも最終的に同一の Traefik Pod(namespace: `kube-system`)へ到達するが、Traefik の中では別々の entrypoint として扱わ れ、互いに干渉しない(図 2.1 に示す)。

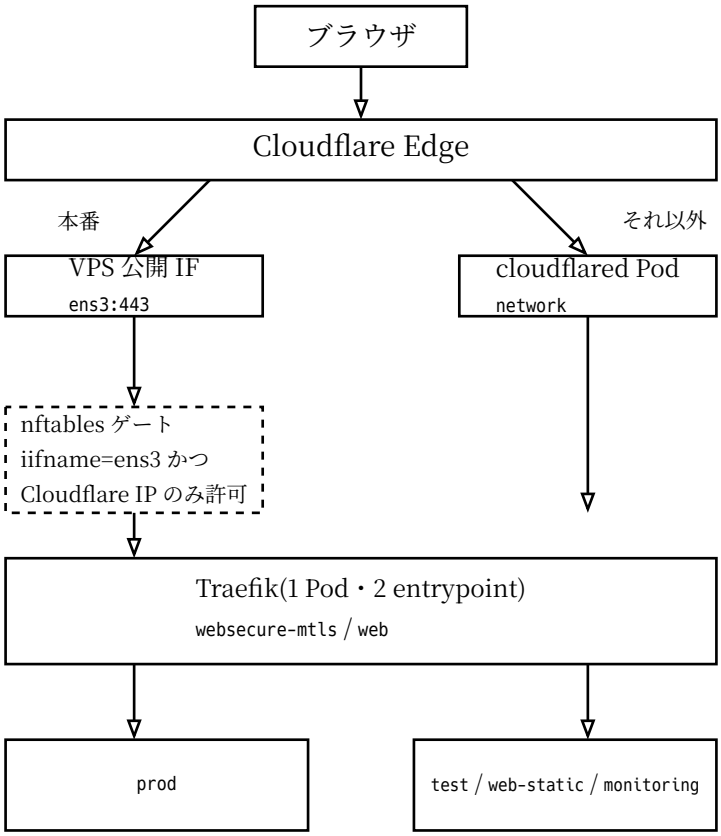


図 2.1：本番(mTLS Origin Auth)とそれ以外(Cloudflare Tunnel)、2 つの入口経路

図 2.1 の通り、`websecure-mtls` だけがホストポートに紐づいた entrypoint であり、`web` にはホ ストへの bind が一切ない。したがって `web` 側に残された IngressRoute は、たとえ nftables のフィルタやクラ イアント証明書の検証が何らかの理由で回避されたとしても、インターネットからは構造的に到達不可能であ る(§7 障害と設計判断も参照)。

段階	本番 — mTLS Origin Auth	それ以外 — Cloudflare Tunnel
対象ホスト	nara-kosensai.com / www.nara-kosensai.com	test.* / storybook.* / grafana.* / pgadmin*.* / info-check.*
Cloudflare エッジ	SSL/TLS: Full (strict) + Authenticated Origin Pulls	Cloudflare Tunnel(通常の DNS/SSL モード設定とは独立に動作)
VPS への入口	公開インターフェース ens3:443(hostPort)	cloudflared Pod(namespace: network)、VPS 側からの outbound 接続のみ
フィルタ	nftables raw prerouting：iifname == ens3 かつ tcp dport 443 かつ saddr ∉ Cloudflare IP → drop(無応答)	該当なし(公開ポートを一切持たないため到達手段自体が存在しない)
Traefik entrypoint	websecure-mtls ・hostPort 443 ・サーバ証明書は Origin CA 証明書 ・ TLSOption(RequireAndVerifyClientCert)でクライアント証明書を検証	web ・ClusterIP :80 のみ ・ホストへの bind なし
到達先 namespace	prod	test / web-static / monitoring

表 2.1：2 つの入口経路の対比

ソースコード 2.1：Cloudflare IP allowlist(nftables、k3s/cloudflare-ip-allowlist.sh が生成。IPv4/IPv6 で独立したテーブルを持つ)

```nft
table ip cloudflare_mtls {
  set cf_ipv4 { type ipv4_addr; flags interval; elements = { ... } }
  chain prerouting {
    type filter hook prerouting priority raw; policy accept;
    iifname "ens3" tcp dport 443 ip saddr != @cf_ipv4 drop
  }
}
table ip6 cloudflare_mtls6 {
  set cf_ipv6 { type ipv6_addr; flags interval; elements = { ... } }
  chain prerouting {
    type filter hook prerouting priority raw; policy accept;
    iifname "ens3" tcp dport 443 ip6 saddr != @cf_ipv6 drop
  }
}
```

冪等な全置換方式(新レンジ取得 → 一時ファイル組み立て → 旧テーブル削除 → nft -f で アトミック適用)で動作するため、curl 失敗時は旧ルールが残るフェイルセーフ側に倒れるが、テーブル削除後の nft -f 自体が失敗すると次回 timer 実行まで drop ルールが不在になりうる (既知の課題、operations.md 参照)。

上記 2 経路を時系列で見たシーケンス図を図 2.2 に示す。

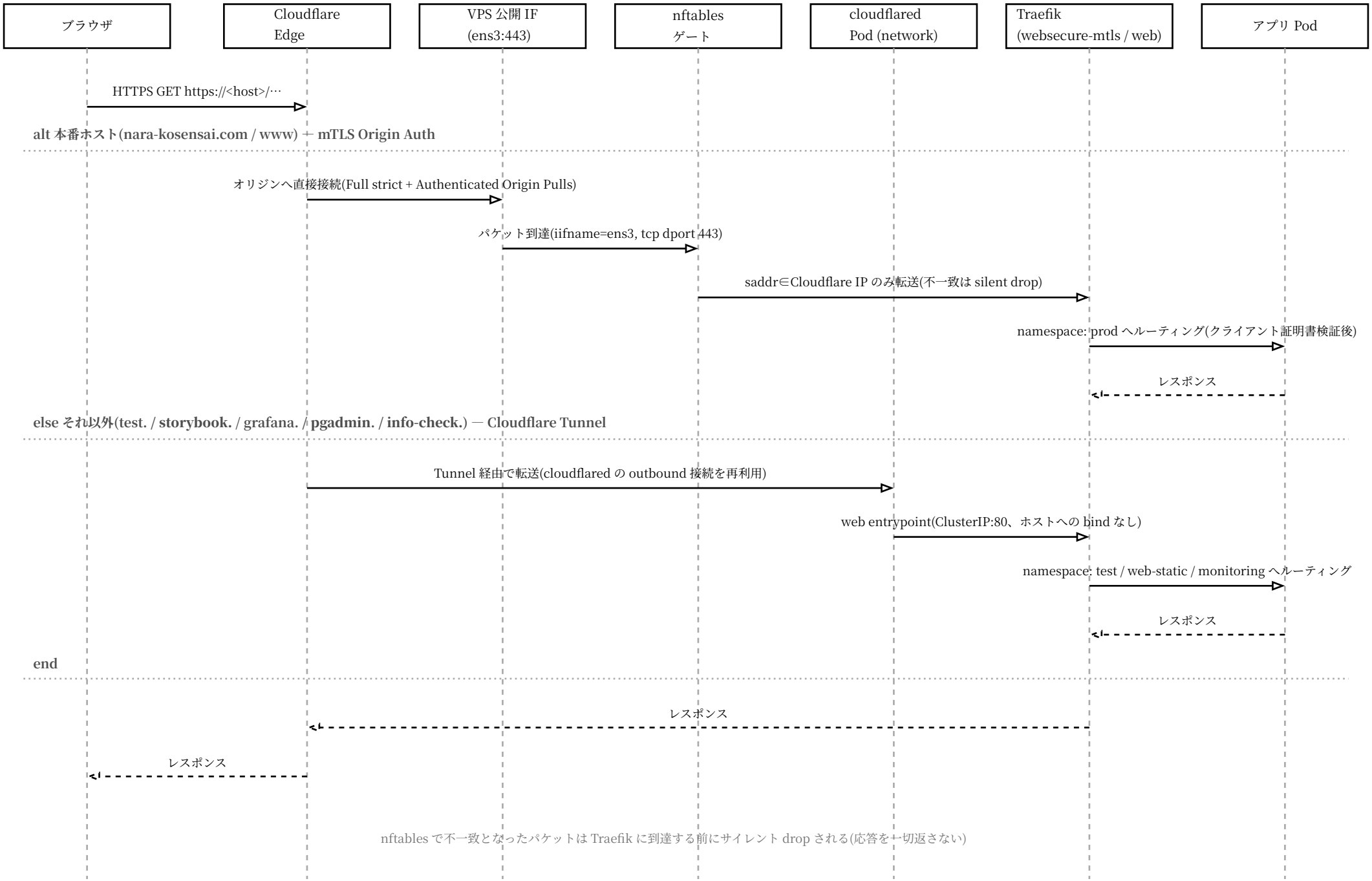


図 2.2：リクエスト処理のシーケンス

2.2 namespace 構成

namespace	代表ホスト	内容
prod	nara-kosensai.com / www	frontend・backend・db (StatefulSet)・website・pgadmin
test	test.nara-kosensai.com	frontend・backend・db・website・pgadmin
web-static	storybook.* / info-check.*	www-archive(過去年度アーカイブ)・storybook・form-preview
monitoring	grafana.nara-kosensai.com	grafana・ loki・ prometheus・ node-exporter・ kube-state-metrics
network	—	cloudflared(公開 Web 用 Tunnel)
kube-system	—	Traefik(k3s 同梱)・CoreDNS

表 2.2：namespace 一覧

管理元は prod/test が 2026kosensai リポジトリの CI、それ以外が manifest リポジトリの CI。web-static/prod/test の IngressRoute は、過去年度アーカイブ(web-static namespace)の Service を cross-namespace で参照する。単一テナント運用のクラスタであるため、Traefik の providers.kubernetesCRD.allowCrossNamespace: true を有効にしている。

2.3 Ingress ルーティング(旧 nginx からの移植)

旧 nginx default.conf のパス単位ルーティング・リダイレクトは、Traefik の IngressRoute/ Middleware として 2026kosensai リポジトリに移植した。nginx の longest-prefix-wins 相当を 明示的な priority(数値が大きいほど高優先)で再現している(表 2.3)。

priority	マッチ条件(prod)	Middleware	転送先
100	Host(www.*)	www-to-apex	apex へ 301
90	/ /2026 /2026/app	redirect-root 等	301
80	PathPrefix(/2026/app/)	security-headers	frontend :3000
70	/2009..2016, /2021	old-years-redirect	学校公式サイトへ 301
60	PathPrefix(/2026/)	security-headers(test は strip-2026-prefix も)	website :80
10	PathPrefix(/)catch-all	security-headers, archive-404	www-archive :80

表 2.3：prod IngressRoute のルーティング優先度

strip-2026-prefix は test の website Service のみに必要(素のリバースプロキシ転送のため)。 archive-404 は catch-all 到達を 404 として web-static の www-archive が持つ /2025/404.html を返す errors Middleware。security-headers は CSP 等を全ホスト共通で 付与する。pgAdmin は webentrypoint(Tunnel 経由)のみを使い、Cloudflare Access で保護する (Traefik 側の Middleware ではない)。

cloudflared(namespace: network)の ConfigMap は webentrypoint 行きの全ホストを一律 http://traefik.kube-system.svc.cluster.local:80 へ転送し、ホスト単位の実際の振り分けは 各 namespace の IngressRoute に委ねている。本番ホスト(nara-kosensai.com / www)は Origin Authentication 移行完了時点でこの ConfigMap の ingress エントリから削除済み。

第 3 章　ネットワーク・セキュリティ

3.1　公開ポート

VPS の公開インターフェースに bind されるポートは 443/tcp のみ。80/tcp は Traefik の `web` entrypoint が ClusterIP のみで待ち受けており、ホストには一切 bind されない。

443/tcp へのアクセス制限は、VPS プロバイダ側とホスト側で役割が異なる。

(1) **VPS プロバイダのファイアウォール**：ポート単位の制限のみ(443/tcp と管理用 SSH 以外を閉じる)。送信元 IP アドレスによるフィルタリングは行っていない。

(2) **ホスト側 nftables**(`k3s/cloudflare-ip-allowlist.sh`、ソースコード 2.1)：送信元 IP アドレスの フィルタリングはこの層のみが担う。Cloudflare が公開している IPv4/IPv6 レンジ (`https://www.cloudflare.com/ips-v4`, `ips-v6`)を、raw prerouting 段階でパケットの 送信元インターフェース(`iifname`、デフォルトルートから動的に解決)と組み合わせて判定する。マッチしない場合は TCP ハンドシェイクにも TLS アラートにも至らずサイレントに drop する。IPv4/IPv6 それぞれ独立したテーブルを持つ。systemd timer(`cloudflare-ip-allowlist.timer`) が日次でレンジを再取得し、`manifest` リポジトリへの push 時にも CI が都度再実行する。

Cloudflare IP アローリストの実体は完全に nftables 側にある点に注意(VPS プロバイダ側の ファイアウォールは補完しない)。

3.2　TLS / mTLS

- **オリジンサーバ証明書**：Cloudflare Origin CA 証明書。SAN は `nara-kosensai.com` と `*.nara-kosensai.com` をカバー(有効期限 2041 年)。`2026kosensai` リポジトリの `production-vps` environment の GitHub Environment Secret(`CF_ORIGIN_CERT` / `CF_ORIGIN_KEY`)として保管し、CI が `prod` namespace に `cloudflare-origin-cert` (`kubernetes.io/tls`)Secret として投入する。
- **クライアント証明書検証**：Traefik の `TLSOption`(namespace: `network`、名前 `cloudflare-mtls`)が、Cloudflare が公開している Authenticated Origin Pulls 用 CA 証明書 (`k8s/base/network/cloudflare-aop-ca.yaml`、公開情報のため git 管理)に対して `RequireAndVerifyClientCert` を要求する。Free プランのため Cloudflare 管理の zone-level 証明書を使用する(per-hostname のカスタムクライアント証明書は非対応)。

ソースコード 3.1：prod IngressRoute の TLS 設定(2026kosensai `infra/k8s/overlays/prod/ingress/ingressroute.yaml`)

```yaml
spec:
  entryPoints:
    - websecure-mtls
  tls:
    secretName: cloudflare-origin-cert
    options:
      name: cloudflare-mtls
      namespace: network
```

3.3　RBAC

Prometheus と kube-state-metrics は、それぞれ専用の ServiceAccount で動作し、read-only の `get/list/watch` のみを許可する ClusterRole を持つ(`k8s/base/monitoring/prometheus-rbac.yaml`、`kube-state-metrics.yaml`)。既定の ServiceAccount には何の権限も付与しない。

kube-state-metrics 用 ClusterRole は `list/watch` のみだが対象に `secrets` を含む(存在・メタデータの可視化目的、書き込み権限はなし)。Prometheus 用はさらに kube-apiserver/ kubelet/cAdvisor のメトリクスエン

ドポイントに ServiceAccount トークンで認証する (`kubelet` ジョブのみ TLS 検証を `insecure_skip_verify: true` で緩めている)。

第 4 章　シークレット管理

Infisical(自己ホスト)は 2026-08-29 付けで完全に廃止した。現在の方針を表 4.1 に示す。

対象	保管場所	理由
DB / pgAdmin / CORS 等 (app-env)	manifest リポジトリ直下の prod.env / staging.env (平文コミット)	DB は ClusterIP 限定で 外部到達不能
GHCR pull トークン	GitHub Environment Secret (production-vps / test-vps)	CI からのみ必要
Cloudflare Origin CA 証明書	GitHub Environment Secret (production-vps、環境限定)	本番専用の機微情報
Cloudflare Access クライアント資格情報	GitHub Actions Secret (管理用 SSH Tunnel 専用)	app-env とは無関係
Grafana 管理者パスワード等	k8s Secret(grafana-secrets) ブートストラップ時に手動投入	gitignore 対象、 循環依存を避けるため
Cloudflare Tunnel 資格情報	k8s Secret(cloudflared-credentials) ブートストラップ時に手動投入	CLI 操作でのみ得られる credentials.json
GHCR pull トークン (web-static 用)	k8s Secret(ghcr-pull) ブートストラップ時に手動投入	private イメージの pull 用

表 4.1：シークレットの保管場所と方針

manifest リポジトリの CI が prod.env / staging.env を VPS の/home/github/app-env/に同期 し、2026kosensai リポジトリの CI がそのパスを直接--from-env-file で参照する。このため manifest 側の CI が 2026kosensai 側より先に一度動いている必要がある。prod/test の ghcr-pull は 2026kosensai の CI 自身がデプロイの都度 GHCR_PULL_TOKEN から再生成する (こちらは一度きりの手動投入ではない)。

第 5 章　CI/CD パイプライン

リポジトリ / ブランチ	処理内容
manifest → main	k3s ブートストラップマニフェスト再配置 → Cloudflare IP allowlist 再実行 → kubectl apply -k k8s/overlays/shared → 各 Deployment の rollout 確認
2026kosensai → prod	backend/frontend/website/storybook 全イメージを無条件で build/push(path-filter なし)→ GHCR pull トークン・Origin CA 証明書を GitHub Environment Secrets から取得 → kubectl apply -k infra/k8s/overlays/prod → rollout 確認
2026kosensai → test	dorny/paths-filter で変更検知(backend/frontend/website)→ 変更分のみ build/push・ apply → .../test/ingress は毎回無条件 apply → rollout 確認 (pgAdmin 用 Cloudflare Access は別途手動設定)
2026kosensai → PR	pr-check.yml：Node 側(pnpm+Turborepo)の lint typecheck build、Go 側の go build / go test。デプロイなし

表 5.1：CI/CD パイプライン一覧

いずれも GitHub Actions から VPS へは、管理用の Cloudflare Access Tunnel(cloudflared access tcp) 経由で SSH 接続する。これは公開 Web 用の Tunnel(namespace network の cloudflared)とは別物で、VPS 側からの outbound 接続であるため、§3 の nftables allowlist とは無関係に動作する。storybook イメージは deploy-prod.yml が build/push するが、web-static への反映(kubectl apply と rollout restart)は manifest 側 CI が担う(prod/test 共有 singleton のため)。

第 6 章　監視

Prometheus(namespace `monitoring`)は次をスクレイプする。

- `node-exporter`：ノードの CPU/メモリ/ディスク
- `traefik`：Traefik 自身のメトリクス(現状 9100 番ポートが未公開のため要修正、既知の課題)
- `kube-state-metrics`：Pod/Deployment/StatefulSet のオブジェクト状態 (Running/Restart 回数/Ready レプリカ数)
- `kubernetes-apiservers`：kube-apiserver のリクエストレイテンシ・エラー率
- `kubelet` / `kubernetes-cadvisor`：ノード・Pod 単位のリソース使用量

いずれも `kubernetes_sd_configs` による in-cluster discovery と、ServiceAccount トークン (`/var/run/secrets/kubernetes.io/serviceaccount/`)による認証を使う標準的な構成。Grafana の Prometheus/Loki データソースは自動プロビジョニングしておらず、初回のみ手動追加が必要 (`http://prometheus:9090`, `http://loki:3100`、いずれも `monitoring` namespace 内)。ログは Grafana Alloy(DaemonSet、`k3s/alloy-logging.yaml`)が全 namespace の Pod ログを Loki へ出荷する。

コンポーネント	Retain?	備考
`prometheus`(10Gi)	されない	メトリクスは再収集可能
`loki`(10Gi)	されない	`retention: 72h`
`grafana`(5Gi)	**される**	ダッシュボード等は再構築コスト高

表 6.1：監視コンポーネントの PVC と reclaimPolicy

node-exporter は `hostNetwork: true` / `hostPID: true` の DaemonSet で旧 `network_mode: host` 相当を再現。Grafana は GitHub OAuth ログインを有効化し、allow-list 外の GitHub アカウントは サインアップできない。

第 7 章　障害と設計判断

7.1　nftables ルールのインターフェーススコープ(2026-08-29)

Origin Authentication 導入時に追加した nftables ルールが、当初 `tcp dport 443` のみで判定してお り、送信元インターフェースを指定していなかった。クラスタ内部には `kubernetes.default.svc` (ClusterIP、`10.43.0.1:443`)を含む複数の ClusterIP がたまたま 443 番を使っており、このルールは Pod→API サーバー間の内部通信まで巻き込んで drop してしまった。

結果として `helm-install-traefik` ジョブ(Traefik の新しいエントリーポイント設定を適用するための Helm インストール)が API サーバーに到達できず、インストール失敗→アンインストール→再インストール のクラッシュループに陥り、Traefik 自体が消滅。本番環境が約 1 時間ダウンした(Cloudflare エラー 521/526)。

対処：ルールを `iifname`(ノードの実際の外部 NIC。ハードコードではなく `ip route show default` から都度動的に解決)でスコープした(ソースコード 2.1)。クラスタ内部のトラフィックは定義上この物理イン ターフェースを通らないため、どの ClusterIP が 443 番を使っていても衝突しない。

同時に、Traefik のコンテナポート・Service 公開ポートでも類似の衝突が発生していた (`websecure-mtls` を chart のデフォルト `websecure` と同じ 8443 番/443 番で定義していたため)。コンテ ナポートは 8444 番、Service 公開は `expose.default: false`(hostPort のみ、Service 経由の公開はそも そも不要)に変更して解消した。

7.2　VPS 再起動によるホスト名変化と k3s ノード名の固定(2026-08-28)

VPS プロバイダのメンテナンス再起動で OS の静的ホスト名が `x162-43-87-201` から `kosensai` に 変化し、k3s が同じ物理マシンを**別のノード**として新規登録した。既存 PersistentVolume は すべて `nodeAffinity` で旧ノード名を固定的に参照(作成後変更不可)していたため、PVC を使う 全 Pod(`db`, `loki`, `grafana`, `prometheus` 等)が新ノードにスケジュールできず `Pending` のまま止まった。

対処：9 個の PV 再作成は本番データを巻き込むリスクが高いため、`/etc/systemd/system/k3s.service` の `ExecStart` に`--node-name x162-43-87-201` を追加し、**k3s 側のノード名を OS ホスト名から独立させて固定した**。以後 OS ホスト名が変わってもノード名は 不変のため、既存 PV の `nodeAffinity` と食い違わない。詳細は `operations.md`「2.」を参照。

第 8 章　リファレンス

項目	値
Kubernetes ディストリビューション	k3s(シングルノード、`--disable=servicelb`)
k3s ノード名	`x162-43-87-201` に固定(OS ホスト名から独立)
Ingress コントローラ	Traefik 3.7(k3s 同梱 Helm チャート)
CNI	flannel(k3s 標準)
ストレージ	local-path(既定、Delete)、local-path-retain(Retain、DB/backend/grafana 用)
公開ポート	443/tcp のみ(80/tcp は ClusterIP 限定で非公開)
Traefik `websecure-mtls`	containerPort 8444, hostPort 443, expose.default: false
Traefik `web`	containerPort 8000 相当(chart 既定), ClusterIP:80 のみ
管理用 SSH	Cloudflare Access Tunnel(公開 Web 用 Tunnel とは別物、outbound-only)
Prometheus/Grafana/Loki	v2.51.0 / 11.0.0 / 3.0.0
node-exporter/kube-state-metrics	v1.8.0 / v2.13.0
Loki 保持期間	72h
関連リポジトリ	`manifest`(本リポジトリ)、`2026kosensai`、`www`、`kosensai-form-preview`

表 8.1：設定値リファレンス